```tsx
"use client";

import React, {
  useEffect,
  useRef,
  createContext,
  useContext,
  forwardRef,
  useMemo,
} from "react";
import { Circle } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🚀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// 🛢️ LOCAL TYPE ISOLATION GATEWAY
export interface RadioProps extends Omit<
  React.InputHTMLAttributes<HTMLInputElement>,
  "onChange" | "value" | "type"
> {
  label?: string;
  error?: string;
  value: string;
  checked?: boolean;
  onChange?: (checked: boolean, value: string) => void;
}

export interface RadioGroupProps {
  value: string;
  onChange: (value: string) => void;
  disabled?: boolean;
  error?: string;
  children: React.ReactNode;
  className?: string;
  orientation?: "horizontal" | "vertical";
}

interface RadioGroupContextValue {
  value: string;
  onChange: (value: string) => void;
  disabled?: boolean;
  error?: string;
  orientation?: "horizontal" | "vertical";
}

const RadioGroupContext = createContext<RadioGroupContextValue | null>(null);
```

```
export const PremiumRadio = forwardRef<HTMLInputElement, RadioProps>(
  (
    { label, error, value, checked, onChange, className, disabled, ...props },
    ref,
  ) => {
    const inputRef = useRef<HTMLInputElement>(null);
    const groupContext = useContext(RadioGroupContext);
    const isInGroup = !!groupContext;
    const isGroupDisabled = groupContext?.disabled ?? false;
    const isGroupError = groupContext?.error;
    const groupValue = groupContext?.value ?? "";
    const groupOnChange = groupContext?.onChange;
    const orientation = groupContext?.orientation ?? "vertical";

    const isActuallyDisabled = disabled || isGroupDisabled;
    const isChecked = isInGroup ? groupValue === value : (checked ?? false);
    const hasError = error || isGroupError;

    useEffect(() => {
      if (inputRef.current) {
        inputRef.current.checked = isChecked;
      }
    }, [isChecked]);

    const handleChange = () => {
      if (isActuallyDisabled) return;

      const newValue = value;

      if (isInGroup && groupOnChange) {
        groupOnChange(newValue);
      }

      onChange?.(true, newValue);
    };

    const radioClasses = cn(
      "relative inline-flex items-center justify-center",
      "w-5 h-5 shrink- shrink- shrink-0",
      "rounded-full border-2",
      "border-border bg-background",
      "transition-all duration-200 ease-out",
      "active:scale-95",
      "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 focus-visible:ring-offset-background",
      "disabled:pointer-events-none disabled:opacity-50
disabled:cursor-not-allowed",
      "aria-invalid:border-destructive aria-invalid:ring-3
aria-invalid:ring-destructive/20",
```

```
  isChecked
    ? "border-brand-primary bg-background shadow-sm hover:shadow-md"
    : "hover:border-brand-primary/50 hover:bg-brand-primary/5",
  hasError && "border-destructive focus-visible:ring-destructive",
  className,
);

const innerDotClasses = cn(
  "transition-all duration-200 ease-out",
  isChecked ? "scale-100 opacity-100" : "scale-0 opacity-0",
);

const labelClasses = cn(
  "text-sm font-medium leading-none",
  "transition-colors duration-200 ease-out",
  "text-text-main",
  hasError && "text-destructive/90",
  isActuallyDisabled && "opacity-50",
  orientation === "horizontal" ? "ml-2" : "ml-2",
);

const containerClasses = cn(
  "inline-flex items-center cursor-pointer select-none",
  isActuallyDisabled && "opacity-50 pointer-events-none cursor-not-allowed",
  orientation === "horizontal" ? "flex-row" : "flex-row",
);

return (
  <label className={containerClasses}>
    <input
      ref={(el) => {
        inputRef.current = el;
        if (ref) {
          if (typeof ref === "function") {
            ref(el);
          } else {
            ref.current = el;
          }
        }
      }}
      type="radio"
      value={value}
      checked={isChecked}
      disabled={isActuallyDisabled}
      onChange={handleChange}
      aria-checked={isChecked}
      aria-invalid={hasError ? "true" : "false"}
      aria-disabled={isActuallyDisabled}
      aria-required={props.required}
      className="sr-only peer"
```

```
          {...props}
        />
        <div className={radioClasses} aria-hidden="true">
          <Circle
            className={cn("w-2.5 h-2.5", innerDotClasses)}
            aria-hidden="true"
          />
        </div>
        {label && <span className={labelClasses}>{label}</span>}
      </label>
    );
  },
);
PremiumRadio.displayName = "PremiumRadio";

export const PremiumRadioGroup = ({
  value,
  onChange,
  disabled,
  error,
  children,
  className,
  orientation = "vertical",
}: RadioGroupProps) => {
  const contextValue = useMemo(
    () => ({
      value,
      onChange,
      disabled,
      error,
      orientation,
    }),
    [value, onChange, disabled, error, orientation],
  );

  const groupClasses = cn(
    "inline-flex",
    orientation === "horizontal" ? "flex-row gap-6" : "flex-col gap-2",
    className,
  );

  return (
    <RadioGroupContext.Provider value={contextValue}>
      <div
        className={groupClasses}
        role="radiogroup"
        aria-invalid={!!error}
        aria-disabled={disabled}
        aria-orientation={orientation}
      >
```

```
        {children}
        {error && (
          <p
            className={cn(
              "text-sm",
              "text-destructive/90",
              "transition-colors duration-200 ease-out",
              orientation === "horizontal" ? "ml-10 mt-1" : "mt-1 ml-7",
            )}
            role="alert"
            aria-live="polite"
          >
            {error}
          </p>
        )}
      </div>
    </RadioGroupContext.Provider>
  );
};
PremiumRadioGroup.displayName = "PremiumRadioGroup";

export { RadioGroupContext };
```